

- 入门简介
 - MIPS汇编
 - 汇编语言
 - MIPS汇编
 - MARS(MIPS ASSEMBLER AND RUNTIME SIMULATOR)
 - 基础知识测试
 - 寄存器简介
 - 什么是寄存器?
 - MIPS中的32个通用寄存器
 - 三个特殊寄存器
 - CP0寄存器
 - 寄存器的功能测试
 - 参考资料
 - MIPS指令集文档
 - MARS相关
 - JAVA运行环境
 - 参考书目
- MIPS指令集架构
 - 认识MIPS汇编指令
 - 初识指令
 - 指令的格式
 - 汇编入门测试
 - 机器码指令
 - 认识机器码
 - 机器码的指令格式
 - 结合手册读懂指令
 - 常用符号详解
 - 指令解读示例
 - MIPS指令手册解读测试
 - 扩展指令和伪指令
 - 扩展指令(PSEUDO INSTRUCTIONS)
 - 伪指令(DIRECTIVES)
 - 扩展指令与伪指令测试
- MARS的获取与使用
- MIPS汇编程序解析
 - 第一个汇编程序
 - 一个简单的循环例子

- [在MIPS中使用数组](#)
- [在MIPS中使用二维数组](#)
- [求Fibonacci数的程序](#)
- [变量声明与定义](#)
- [MIPS汇编程序设计](#)
- [测试程序设计（本章请将开始搭建CPU时在进行阅读）](#)

入门简介

MIPS汇编

汇编语言

由于硬件方面原因，计算机中的数据都是以**二进制**的形式表示的，这也代表着，CPU所直接处理的都是一条条二进制**机器码**指令。而这些单纯的机器码是很难以阅读和理解的，因此人们发明了比机器语言稍微高级的一种程序语言——**汇编语言**(Assembly Language)。事实上，与其说汇编语言是一种语言，不如说它是一种**助记符**——用一些易于理解的符号代表特定含义的机器码，用**标签**(Label)来代替地址。

相比于C、Java这些高级语言，汇编语言要低级的多，他和机器码几乎是一一对应的，一般用于**贴近于硬件**的编程，正如我们的计组实验。我们在这里学习汇编语言，一方面是为了让同学们通过学习汇编语言来理解计算机底层硬件的**运行过程**，另一方面是使用汇编语言来编写**测试程序**验证我们设计的CPU的正确性。

由于汇编语言涉及到的知识点众多，部分知识点十分琐碎，建议在**第一遍学习时不要陷入细节问题之中**，快速阅览整章，对汇编语言有一个大概认识之后再回过头来将**第一遍学习中不理解**的点逐个攻克。

MIPS汇编

不同架构的CPU都有各自独特的汇编语言，例如我们将会使用的MIPS汇编语言就是专门用于**MIPS体系结构**CPU的汇编语言。MIPS汇编的最大特点就是指令的**结构比较单一**，所有的指令都可以翻译成**32位二进制机器码**，指令的组成也都只有**R,I,J**这三类(依照格式进行的分类，后面会详细讲解)。

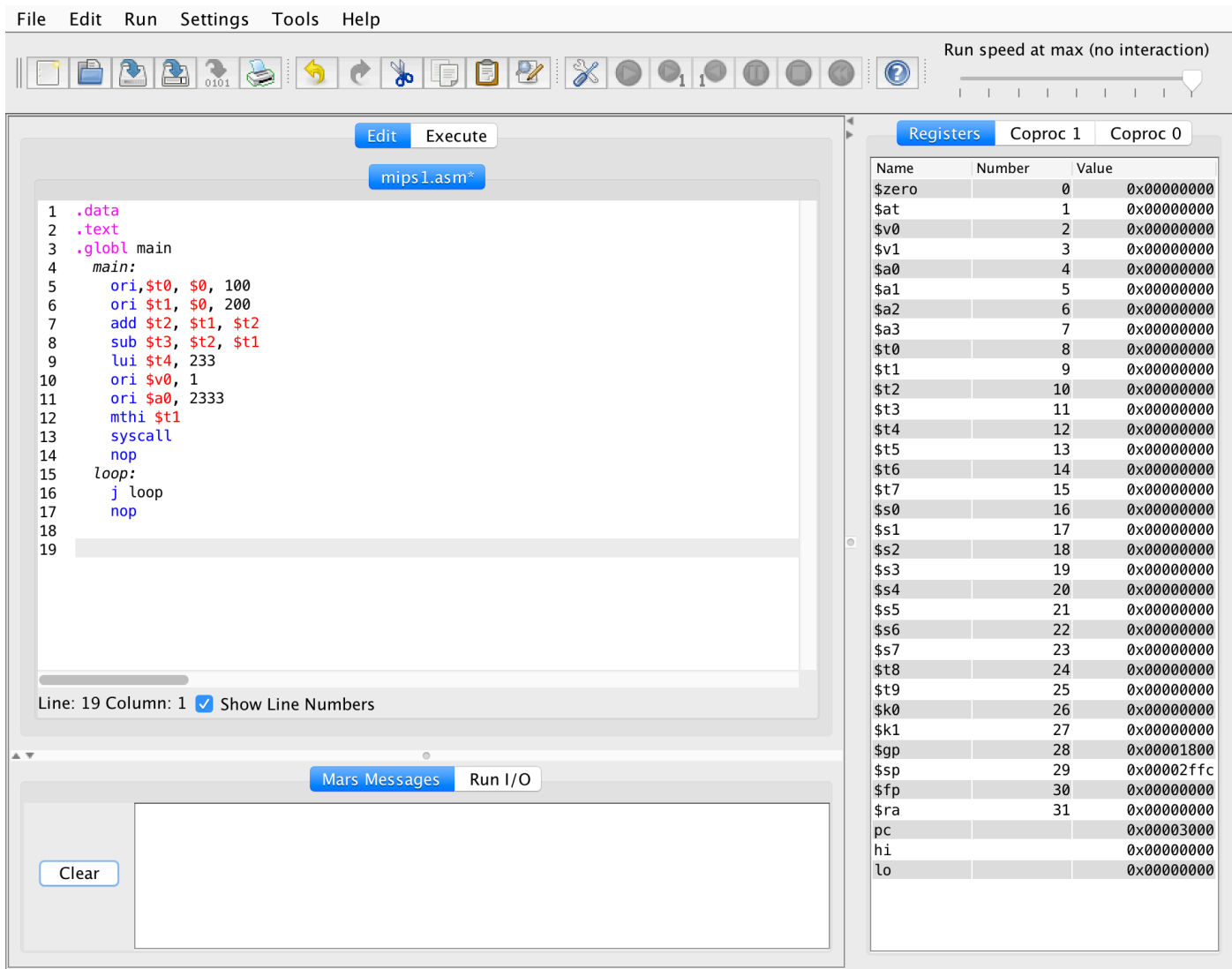
这一特点让他看起来有些“死板”，因为大多数指令并不需要使用32位机器码才能表示，一般10位、20位就已经足够了。从这点来看MIPS汇编确实比较**浪费储存空间**，这也是MIPS架构CPU不被主流所喜得重要原因。不过，有弊就有利，MIPS汇编这种固定的指令格式非常具有规律性。可想而知，相对于其他架构的CPU，MIPS架构CPU得设计难度相对较低，很时候我们这些初学者。而它的这些规律性，会使我们在设计MIPS体系结构的CPU的过程中，更充分地体会到**工程化设计**带给我们的便利，更容易理解CPU的整体构造。

MARS(MIPS ASSEMBLER AND RUNTIME SIMULATOR)

MARS是一个轻量级的、用于教学的MIPS汇编语言**集成开发环境(IDE)**。它由密苏里州立大学开发，基于**java环境**运行，完全绿色且免费。我们的课程及使用此软件来进行MIPS汇编语言的学习、编写和运行。

在我们的实验中，选择Mars的原因主要有以下几点：

- Mars是用于MIPS汇编语言的IDE，和我们设计的CPU相同。
- Mars的Help文档中，对指令、伪指令、系统调用等各种功能都有详细的介绍与使用样例，便于查阅使用。
- 无论是正常程序还是异常处理程序，Mars上都能很好的运行，同时还能够按需求导出程序代码运行于我们的CPU程序中。
- 最重要的是，Mars中的指令都是在它内部的CPU模型上运行的，无论单周期还是流水线，它内部的CPU模型都是我们实验设计的黄金校验模型，对我们的CPU设计有很大的指导作用。



图为MARS界面

基础知识测试

寄存器简介

什么是寄存器？

寄存器是CPU的组成部分之一，是一种**高速存储器**(甚至是CPU可以使用的最高速的一种存储器)，可以用来暂存指令，数据，地址等。由于寄存器的成本较高，一般的CPU中又有数量有限的寄存器可供使用。

在MIPS体系结构中，CPU对数据的操作均是基于寄存器的。内存中的数据需要先使用读取(load)类指令保存到寄存器中才可使用。操作完成的数据也不能直接保存到内存中，需要使用装载(store)类指令保存至内存中。

在我们是严重的CPU是32位CPU，一次能处理的最大位数即为32位，绝大部分寄存器也均是1字大小的(即4字节，也就是32位)。

MIPS中的32个通用寄存器

在一般(至少是本课程所需要用的)的MIPS汇编程序中，比较常用的是**32个通用寄存器**。所谓通用寄存器，代表它们没有明确规定的用途，程序员可以随意对他们赋值、取值、同时它们的值也可以直接参与到各种指令之中。然而，虽然冠有通用的头衔，程序员们还是会以一定的规则来使用他们，这是为了便于程序员之间的交流，同时也是为编译器等工具定下了一定的标准。

这32个通用寄存器有两种命名方式，一种是按序号命名 **\$0~\$31**(注意前面有个美元的符号)，一种就是按功能命名，如下表：

寄存器编号	寄存器名称	用途说明
\$0	\$zero	常量0
\$1	\$at	保留给汇编器使用的临时变量
\$2-\$3	\$v0-\$v1	函数调用返回值
\$4-\$7	\$a0-\$a3	函数调用参数
\$8-\$15	\$t0-\$t7	临时变量
\$16-\$23	\$s0-\$s7	需要保存的变量
\$24-\$25	\$t8-\$t9	临时变量
\$26-\$27	\$k0-\$k1	留给操作系统使用
\$28	\$gp	全局指针
\$29	\$sp	堆栈指针
\$30	\$fp	帧指针
\$31	\$ra	返回地址

其中usage那一列可能暂时不好理解，没有关系，在之后的学习中会慢慢用到。特别地，对于 **\$0**和 **\$1**两个寄存器：

- **\$0**一般不能用于赋值，但这不代表你对他赋值就是错的。在实际中，你可以对 **\$0**进行赋值，但他的值会始终保持为0.也就是说对 **\$0**赋值并不违反语法，但没有实际效果。

- **\$1**保留给汇编器，一般不使用此寄存器(使用该寄存器可能会导致难以发现的bug，具体原因请在编写MIPS时注意扩展指令如何翻译成普通指令的)。

三个特殊寄存器

- **PC(Program Counter)**: 它用于存储当前CPU正在执行的指令在内存中的地址。需要注意的是，这个寄存器的值不能用常规的指令来进行取值和赋值，但这并不意味着不能做到，只是麻烦一些。那么，怎样对其取值、赋值呢？这可以作为一个思考的题目，在之后的学习中自行探索。(提示：跳转)
- **HI**: 这个寄存器用于乘除法。它被用来存放每次乘法结果的**高32位**，也被用来存放除法结果的**余数**。
- **LO**: HI的孪生兄弟。它被用来存放每次乘法结果的**低32位**，也被用来存放除法结果的**商**。

CP0寄存器

当我们的CPU设计推进到比较深入的阶段时，我们就需要对**异常和中断**进行处理，届时我们就会使用到CP0存储器。

CP0是一个系统控制的**协处理器**，而CP0存储器则是该处理器工作时用到的一些寄存器。在我们的实验中，只会用到其中的4个寄存器：SR、Cause、EPC和PRId。

这里先介绍一下这几个寄存器的职能，具体细节在p7时会详细讲解。

- **SR**: 用于系统控制，决定是否允许异常和中断
- **Cause**: 记录中断和异常的类型
- **EPC**: 保存异常或中断发生时的PC值，也就是发送异常或中断时CPU正在执行的那条指令的地址。当处理完成之后，CPU会根据这个地址返回到正常程序中继续往下执行。
- **PRId**: 处理器Id，用于实现个性的寄存器。

上面所述的那些寄存器是MIPS体系结构和我们的课程中常用的重要的寄存器，他们都是32位的。但寄存器的范围很广，上述的寄存器只是他们的一部分。除他们以外，还有许多寄存器被用于临时使用，或许都没有名字，而他们的真正应用，需要同学们在自己设计CPU时自行合理安排。

寄存器的功能测试

参考资料

MIPS指令集文档

- 中文、精简版: [MIPS-C指令集](#)
- 英文、完整版: [MIPS32 Architecture For Programmers](#)

MARS相关

- MARS v4.5下载: [Mars4_5.jar](#)

JAVA运行环境

- Pracle官方下载: [Oracle官方](#)

参考书目

- 《See MIPS Run Linux》: [《See MIPS Run Linux》](#)

MIPS指令集架构

认识MIPS汇编指令

初识指令

在真正开始掌握MIPS汇编指令之前，我们需要先知道什么是**指令**。指令，即是由处理器指令集架构(Instruction Set Architecture，可以理解为计算机体系结构中对程序相关的部分所作的定义)定义的处理器的独立操作，这个操作一般是运算、存储、读取等。一个指令在CPU中真正的存在形式是高低电平，也可以理解为由01序列组成的机器码。但因为机器码人类难以阅读和理解，所以指令一般由汇编语言来表示，也就是我们俗称的汇编指令。从这个角度上来说，汇编指令只是指令的一种表现形式而已，其实质是一样的。

一条指令的功能较为单一，一般不具有复杂的逻辑，例如【将两个寄存器相加并存入另一个寄存器】，或者是【如果某个寄存器的值满足某个条件则跳转至某条指令】。不过，虽然这些指令很简单，但最终，我们可以用他们组合出丰富多彩、功能强大的程序。

那么，指令究竟长什么样呢？让我们一起来看一段汇编程序：

```
.data
.text
.global main
main:
    addi $t0, $0, 100 # tag
    ori $t1, $0, 200
    add $t2, $t1, $t2
    sub $t3, $t2, $t1
    lui $t4, 233
    ori $v0, 1
    mthi $t1
    ori $a0, 2333
    mthi $t1
    ori $a0, 2333
    mthi $t1
    syscall
    nop
loop:
    j loop
    nop
```

其中tag标记的 `addi $t0, $0, 100` 就是一条指令，它的含义为“将 `$0` 寄存器的值加上 100，并将结果存入 `$t0` 寄存器”。

需要注意的是，虽然其主要有指令构成，但汇编语言并非全部有指令构成。上面的代码中，第5-16行、18-19行为严格意义上的指令，其余由标签、伪指令等。相信看了上面一段代码样例，你已经找出了指令的规律，下面我们就来详细地进行解读。

指令的格式

在MIPS汇编语言中，指令一般由一个指令名作为开头，后跟该指令的操作数，中间由空格或逗号分开。指令的操作数的个数一般为0-3个，没有一个指令都有其固定操作数个数。一般来说，指令的格式如下：

指令名 操作数1， 操作数2， 操作数3

不过，也有如下的指令格式，一般用于存取类指令：

指令名 操作数1， 操作数3(操作数2)

所谓操作数，即指令操作所用的实体，可以是寄存器、立即数或标签，每个指令都有其固定的对操作数形式的要求。而标签最终会由汇编器转换为立即数。所谓立即数，即在指令中设定好的常数，可以直接参与运算，一般长度为16位二进制。而标签，用于使程序更简单清晰。标签用于一个程序跳转的地址(类似于函数名，或者C语言中 `goto` 的跳转目标)。在MIPS汇编中(以及其他大部分汇编语言中)，标签用如下的方式写出：

```
name:
```

其中的name代表这个标签的名称，可以自行取名。

常见的指令格式样例：

```
add $s0, $a0, $a1
addi $s0, $a0, 12
mult $s1, $s2
beq $a1, $a2, -2
blez $s1, -2
jr $ra
j 0x00003014
```

当然，前面说过，可以使用标签来代替某个地址，因此也可以以如下的方式书写：

```
beq $a1, $a2, loop
j loop
```

这里的 `loop` 就是一个标签，它所代表的是一段代码的起始地址。在进行汇编时，汇编器会自动把标签转换成我们所需要的立即数，这样就不用我们自己去计算这些地址偏移量，简化了编程难度。

注意：在MARS中，跳转指令只能使用标签来进行跳转，不能使用立即数！

由此可以看出，在MIPS汇编语言中，操作数的形式并非绝对严格固定的，而是具有一定的灵活度。虽然在MIPS标准指令集中，一条机器码指令的格式是固定的，但汇编器可以将多种形式的汇编指令转换为同样意思的机器码指令。因此，许多指令有比标准写法简单的写法。这部分内容会在后面进行讲解(详见【扩展指令】)，或者也可以自行查阅MARS的帮助文档。

为了更好的理解汇编指令，下一小节将详细讲解MIPS机器码指令，这一汇编指令转换后的形式。

PS: 在本教程中，没有特殊说明的情况下都不需要考虑延迟槽的存在。

汇编入门测试

机器码指令

认识机器码

大家都知道，计算机只能理解二进制形式的数据。而我们前面所说的汇编语言，最终就会转化为机器语言——也就是机器码指令，CPU可以直接识别这种机器语言，从而去完成相应的操作。在我们学习的MIPS汇编中，所有的指令长度均为32位，即4字节，或者说1字。同时，从硬件的角度来讲，每条指令的执行周期大多为1个CPU周期，这在深入学习之后可以更好的理解，这里只是阐述一个概念。因此机器码就是CPU最基本的一种操作，也是原子操作，不可被打断。

所有指令长度均相同，这是精简指令集(RISC, Reduced Instruction Set Computing)的特征，这种指令集包括MIPS和手机中常用的ARM等；与之相对的是复杂指令集(CISC, Complex Instruction Set Computing)，包括PC中常用的x86架构，这种指令集的特点是指令数目多，指令长度并不完全相同。

一段汇编语言可以转换为一段机器码，例如下面这段汇编指令：

```
.data
.text
.global main
main:
    addi $t0, $0, 100
    ori $t1, $0, 200
    add $t2, $t1, $t2
    sub $t3, $t2, $t1
    lui $t4, 233
    ori $v0, 1
    ori $a0, 2333
    mthi $t1
    syscall
    nop
loop:
    j loop
    nop
```

其转换后的结果为(16进制)：

```
20080064
340900c8
012a5020
01495822
3c0c00e9
34420001
3484091d
01200011
0000000c
00000000
08000c0a
00000000
```

机器码的指令格式

32位的机器码需要一定的格式才能被理解。一般来说，在MIPS指令集中，指令分为三种格式: R型、I型和J型。

- R型指令

R型指令的操作数最多，一般用于运算指令。例如 `add,sub,sll` 等。其格式如下(左侧为高位，右侧为低位)：

op	rs	rt	rd	shamt	func
6bits	5bits	5bits	5bits	5bits	6bits

- I型指令

I型指令的特点是有**16位的立即数**(偏移也是一样的道理)。因此，I型指令一般用于 `addi,subi,ori` 等与立即数相运算的指令(这里需要注意：**在写汇编语言的时候，需要使用负号来标记负数，而不要和机器码一样认为首位为一的就代表负数**)，或 `beq,bgtz` 等比较跳转的指令，因为他们要让两个寄存器的值相比并让PC偏移offset这么多，刚好利用了全部的字段。还有存取指令，例如 `sw,lw`，他们在使用时需要对地址指定一个偏移量，也会用到立即数字段。

op	rs	rt	offset or immediate
6bits	5bits	5bits	16bits

- J型指令

J型指令很少，常见的为 **j** 和 **jal**。他们需要直接跳转至某个地址，而非利用当前的PC值加上偏移量计算出新的地址，因此需要的位数比较多。

op	address
6bits	26bits

需要注意的是，严格来说，并非所有的指令都严格遵守上面三种格式，有的如 **eret**, **syscall** 指令一样没有操作数；有的如 **.jalr** 指令一样某些字段被固定为某个值。不过，就大部分指令而言，都可按上面三种格式进行解释，某些字段被固定也可以按照格式来识别为R，I，J中的一种，因此这三种格式要着重理解。

解读:

- **op**: 也称opcode、操作码，用于标识指令的功能。CPU需要通过这个字段来识别这是一条什么指令。不过，由于op只有6位，不足以表示所有的MIPS指令，因此在R型指令中，有func字段来辅助它的功能。
- **func**: 用于辅助op来识别指令。
- **rs,rt,rd**: 通用寄存器的代号，并不特指某一寄存器。范围是 **\$0~\$31**，用机器码表示就是00000~11111。
- **shamt**: 移位值，用于移位指令。
- **offset**: 地址偏移量。
- **immediate**: 立即数。
- **address**: 跳转目标地址，用于跳转指令。

结合手册读懂指令

常用符号详解

- **+**: 《MIPS-C指令集》中所有的 **+** 一律将其左右两侧的操作数视为无符号数，再进行相加。
- **-**: 《MIPS-C指令集》中所有的 **-** 一律将其左右两侧的操作数视为无符号数，再进行相减。
- **下标**: 变量的下标代表变量中所取用的位，*temp*₃₁ 即代表 **temp** 变量在二进制表示下的第31位。特别的，下标中的 **..** 可理解为“至”，*temp*_{31..0} 与 **temp[31:0]** 是等价的。
- **上标**: 某一位二进制数(即0或1)的上标代表该数字重复的位数，0⁵ 即代表 **5'b000000**。

- **||**: 该符号代表拼接操作。temp1||temp2与 {temp1, temp2}等价。
- **sign_extend()**: 代表符号扩展，其扩展结果的正负会与扩展数相同，一般用于有符号数。其扩展方法是，将被扩展数不足的部分用被扩展数的**符号位**补全。即若被扩展数为正数，则将其高位用0补全至相应位数；若为负数，则在高位补1至相应位数。
- **zero_extend()**: 代表无符号扩展，即倘若被扩展数位数不足，则将其高位补0至相应位数。
- **GPR**: 即General Purpose Register(通用寄存器)，其后的中括号内为寄存器编号，GPR[rs]即可表示编号为rs的寄存器。
- **memory**: 表示内存，当其后跟随中括号时，表示储存在以括号中数值为起始地址的**4字节**内存中的数据。

以下部分暂不需要掌握，建议在学习了p5流水线相关知识后再进行学习，pre时建议直接跳过此部分阅读指令解读示例

指令解读示例

下面我们结合文档《MIPS-C指令集》来讲解如何读懂一条机器码指令的功能与格式。下表为 add指令的指令详解：

编码	31	26	25	21	20	16	15	11	10	6	5	0
	special 000000		rs		rt		rd		0 00000		add 100000	
	6		5		5		5		5		6	
格式	add rd, rs, rt											
描述	GPR[rd] ← GPR[rs]+GPR[rt]											
操作	temp ← (GPR[rs] ₃₁ GPR[rs]) + (GPR[rt] ₃₁ GPR[rt]) if temp ₃₂ ≠ temp ₃₁ then SignalException(IntegerOverflow) else GPR[rd] ← temp _{31..0} endif											
示例	add \$s1, \$s2, \$s3											
其他	temp ₃₂ ≠ temp ₃₁ 代表计算结果溢出。 如果不考虑溢出，则 add 与 addu 等价。											

- **编码**: 二进制指令各字段的值和功能。这里可以看出，add指令的opcode为000000；其后跟随着rs,rt,rd三个指示寄存器的字段；之后为全0的shamt字段，最后的func字段值为100000.
- **格式**: 汇编指令的书写格式。其中的rd,rs,rt在真正书写时要替换为具体的寄存器。
- **描述**: 指令的功能。GPrs表示寄存器堆，在此和中括号括起来的寄存器编号结合起来可以理解为某个寄存器。例如 GPR[rd]表示编号为rd的寄存器。add指令的描述表示该指令的功能为将编号为rs和rt的寄存器的值相加，存入编号为rd的寄存器中。

- 操作:** 指令的具体执行过程及细节。 $GPR[rs]_{31} \parallel GPR[rs]$ 表示将rs号寄存器的bit31和rs号寄存器原本的值进行拼接，结合成一个33为的数值。进一步地， $temp \leftarrow (GPR[rs]_{31} \parallel GPR[rs]) + (GPR[rt]_{31} \parallel GPR[rt])$ 表示将这两个33位的数值相加，存入一个临时变量temp中。接着，判断这个临时变量的bit32和bit31进行比较，若不相等，则说明出现了溢出，那么就会引发整数溢出异常(关于**异常**的知识会在以后学到)；如果没有发生溢出，则会将temp的低32位存入rd寄存器中，作为最终的结果。
- 示例:** 汇编指令书写样例。这里的rs,rt和rd已经替换为了具体的寄存器。
- 其他:** 补充说明。这里提到如果不考虑溢出，则add与另一个指令addu等价，这便指出了add的特性：add会检验计算的结果是否会溢出。在一些c语言的编译器中，假发的计算并不会引发异常。例如计算 $2000000000 + 2000000000$ ，得到的结果-2094967296，说明计算出现了溢出，但并未引发异常。这说明该编译器使用的是addu。

下图更加直观地表示了汇编指令转变为机器码指令的过程：

汇编指令转变为机器码

事实上在CPU中，机器码还要按照同样的格式进行解码，并使CPU执行相应的功能。相关的知识会在今后学到。

上面的add指令中，所有的操作数形式均为寄存器，那么当操作数为标签时会怎样呢？事实上，汇编器会将标签翻译为一个立即数，再转化为机器码，下面以另一个指令: beq为例子再进行一次详解。

	31	26	25	21	20	16	15	0
编码	beq 000100		rs	rt		offset		
	6		5	5		16		
格式	beq rs, rt, offset							
描述	if (GPR[rs] == GPR[rt]) then 转移							
操作	if (GPR[rs] == GPR[rt]) PC ← PC + 4 + sign_extend(offset 0 ²) else PC ← PC + 4							
示例	beq \$s1, \$s2, -2							
其他								

其所在的程序如下：

```

.text
label:
    add $s1, $s2, $s3
  
```

```
beq $0, $s1, label
nop
```

操作详解:

可以看到，offset是一个数值，而label是一个标签，因此需要将label转化为具体的数值。根据代码，beq跳转之后的指令应为第三行：add \$s1, \$s2, \$s3。这一条指令是beq的上一条指令，也就是说这两条指令的地址差4.而根据手册，beq跳转的结果为PC + 4 + sign_extend(offset // 0²)，也就是说4 + sign_extend(offset // 0²)。可以算出，offset为0xffff(注意这是一个16位的负数)。因此，最终beq \$0, \$s1, label的转化结果为0x1011ffe。

除了上述的运算指令和跳转指令之外，MIPS汇编中常用的指令还有存取指令，其中存取的指令结构基本相同。下面以最基本的lw指令为例进行解读。

编码	31	26	25	21	20	16	15	0
	lw 100011		base		rt		offset	
	6		5		5		16	
格式	lw rt, offset(base)							
描述	GPR[rt] ← memory[GPR[base]+offset]							
操作	Addr ← GPR[base] + sign_ext(offset)							
	GPR[rt] ← memory[Addr]							
示例	lw \$v1, 8(\$s0)							
约束	Addr 必须是 4 的倍数(即 Addr _{1:0} 必须为 00)，否则产生地址错误异常							

- **编码:** lw是条I型指令，op = 100011，base字段其实就是一般I行指令格式中的rs寄存器字段。
- **格式:** 存取指令的格式与一般I型指令不同：要把基地址base用括号括起来，外面是偏移量offset。
- **操作:** 首先将偏移量offset符号扩展为32位后与base寄存器中的值相加，求出具体的内存地址，然后将该地址中数据提取出来赋给rt寄存器。
- **约束:** 对于存取指令，最终计算出来的地址Addr一般都是有强制约束的，比如lw指令的addr就必须是4的倍数，因为1个字=4个字节；lh(取半字)指令的Addr就必须是2的倍数，因为1个半字=2个字节。这样做可以防止你对内存数据随意进行操作。

MIPs指令手册解读测试

扩展指令和伪指令

为了方便编程，MIPS汇编在标准指令的基础上又提供了许许多多的扩展指令，其中就包括前面提到过的对基本指令的转写(例如用标签代替立即数)。此外还有对基本指令的操作数的略写和使用基本指令组合出新的指令。这类**扩展指令**称为【Pseudo Instructions】。

虽然使用标签、基本指令和扩展指令已经可以写出汇编程序，但其灵活性仍有限制。例如，仅用标签和指令无法初始化变量(数据段中的数据)；无法区分代码属于普通代码或异常处理代码。因此，汇编器提供了**伪指令**(Directives)来让我们指导汇编器的工作。有了伪指令，我们可以声明全局标签、声明宏、设置异常数据段和代码段等等。

扩展指令(PSEUDO INSTRUCTIONS)

学完机器码之后，我们就可以将一条机器码还原为一条汇编指令，例如将0x08000c0a还原为j 0xc0a这种形式，我们可以将其称之为标准指令，或者叫基本指令。也就是说它的每个操作数都和机器码中的相应字段完美对应。然而，完全使用基本指令会大大降低程序的易读性和灵活性，因此便诞生了扩展指令。

扩展指令的功能主要是简化程序。汇编器将一些常用的、但标准指令集不提供的功能**封装**为一条指令；或者改变现有指令的操作数的i形式或个数，使其以新的形式出现。需要注意的是，他们只是**形式上**是一条新指令，而实际上，在汇编器将其汇编之后，还是**使用标准指令来实现的**。

最常用到的一条扩展指令是li指令，它用来为某个寄存器赋值，比如li \$a0, 100就是将100赋给\$s0寄存器。汇编器在翻译这条扩展指令时会根据需要，将它翻译成不同的基本指令或基本指令的组合。如下，第一条li指令后面的立即数不多于16位，因此只被翻译成了一条addiu；第二条li指令后面的立即数多于16位，因此被翻译成了lui+ori的组合。

Address	Code	Basic	
0x00003000	0x20080000	addi \$8,\$0,0	7: la \$t0, fibs
0x00003004	0x200d0030	addi \$13,\$0,48	8: la \$t5, size

上面的例子就是利用现有基础指令组合出新的扩展指令。当然，我们也可以对现有指令进行简化或转写，例如前面提到过的j 0xc0a，我们可以将其立即数改为一个标签，这样汇编器会在进行汇编时，会将标签所代表的立即数计算出来，形成基本指令的形式。例如下图：

0x00003028	0x08000c0a	j 0x00003028	16: j loop
------------	------------	--------------	------------

想要了解更多的扩展指令，可以查看Mars的Help文档中Extended(pseudo)Instructions一栏。

伪指令(DIRECTIVES)

伪指令(Directives)是用来知道汇编器如何处理程序的语句，有点类似于其它语言中的预处理命令。伪指令不是指令，他并不会被编译成机器码，但它却能影响其他指令的汇编结果。常用的伪指令有以下几个：

- **.data**: 用于预先存储数据的伪指令的开始标志。
- **.text**: 程序代码指令开始的标志。
- **.word**: 以字为单位储存数据。
- **.ascii**: 以字节为单位存储字符串。
- **.space**: 申请若干个字节的未初始化的内存空间。

这些指令的详细介绍请参考第4节(MIPS汇编程序解析)第2单元(变量的声明与分配)。想要了解更多的伪指令可以查看Mars的Help文档中Directives一栏。

注：Mars的Help文档写的比较简略，例如他没有指出**.data**和**.text**伪指令后面可以跟着参数。因此优先以教程中的说法为准(也就是说你要去看看【汇编语言与MARS: MIPS汇编程序解析/变量声明与定义】)

扩展指令与伪指令测试

MARS的获取与使用

MIPS汇编程序解析

第一个汇编程序

现在。我们通过几个简单的汇编程序。理解汇编语言的形式和各个指令的用途。同学们可以把这些代码复制到自己的MARS编辑器中，并在自己的电脑上运行一遍，注意MARS调试功能的使用，使得同学们可以更清楚地看到寄存器中数值的变化，明白各个指令的意义。有关循环的一些内容，在后文中会有详细的解释。

下面以一个经典的题目为例，编写我们的第一个汇编程序。

题目内容：输出 “Hello World”

文本代码：

```
.data
str: .asciiz    "Hello World"

.text
la $a0, str
li $v0, 4
syscall

li $v0, 10
syscall
```

这是最简单的一个MIPS汇编程序，同学们可以把这段代码复制到MARS中，在运行之前确保(Mars->Settings->Memory Configuration)为Compact, Data at Address 0。这个选项代表数据(.data)段的地址是从0x0开始，程序(.text)段的地址是从0x3000开始。

下面我们一行一行地分析这段代码。

第1行`.data`，代表变量地生命和分配从这里开始

第2行`str: .asciiz "Hello World"`，代表分配了一定的内存空间，用来储存这个字符串，字符串最后有一个“\0”，str是这个字符串的标签(label)

第4行的`.text`代表程序从这里开始

第5行的`la`，是一个扩展指令，在按F3快捷键之后进入Execute页面，可以看到这一个指令被转换成了`addi $4, $0, 0`；同理，第6行和第8行的`li`，也是一个扩展指令，这一个指令被转换成了`addiu $2, $0, 4`和`addiu $2, $0, 10`

第7行和第10行的`syscall`，会根据当前的`$v0`寄存器的值，进行相应的操作，比如执行第7行`syscall`的时候，`$v0 = 4`，此时就会输出`$a0`寄存器指向的字符串，执行第10行`syscall`的时候，`$v0 = 10`，此时程序运行结束。有关`syscall`的说明，可以按快捷键F1，在MIPS->Syscalls页面中查到相应的用法。

这个MIPS汇编程序在运行的时候，先把str的地址(0x0)赋值给`$a0`，然后令`$v0 = 4`，执行`syscall`，输出`$a0`寄存器指向的字符串“Hello World”，直到遇到“\0”为止，最后令`$v0 = 10`，执行`syscall`，结束程序。

汇编语言与我们之前所学习的C语言还是有很大的区别的，由于汇编语言的主体代码中并没有变量名、函数名、各种运算符和语句块。我们仅能通过有限的32个寄存器以及众多的汇编指令和标签来实现我们之前所学的C语言所能够实现的代码，有兴趣的同学可以自行搜索相关资料进行学习，将来在操作系统和编译这两门课程中你们还将继续与这些汇编程序打交道。

如果认真阅读了上面的代码，想必已经体会到了汇编指令在可读性方面的巨大缺陷，所以请务必为自己编写的汇编程序加上注释，防止代码写完之后自己都看不懂。

一个简单的循环例子

下面以一个简单的题目为例，说明在MIPS中如何使用循环。

题目内容：输入一个数 n ，输出 $1 + 2 + 3 + \dots + n$ (保证 n 和输出结果都在int的范围之内)

文本代码:

```
.text
li $v0, 5
syscall # 输入一个整数，输入的数存到$v0中
move $s0, $v0 # 赋值, $s0 = $v0
li $s1, 0 # $s1用于存储累加器的值, $s1 = 0
li $t0, 1 # $t0 是循环变量

loop:
    bgt $t0, $s0, loop_end # 这里用了一个扩展指令bgt，当$t0 > $s0的时候跳转到loop_end
    add $s1, $s1, $t0 # $s1 = $s1 + $t0
    addi $t0, $t0, 1 # $t0 = $t0 + 1
    j loop # 无条件跳转到loop标签

loop_end:
    move $a0, $s1 # 赋值, $a0 = $s1
    li $v0, 1 # $v0 = 1, 在syscall中会输出$a0的值
    syscall
    li $v0, 10 # $v0 = 10
    syscall # 结束程序
```

在运行之前确保(Mars->Settings->Delayed branching)前面不要打钩，即不考虑延迟，遇到跳转指令立刻跳转，不执行跳转指令之后的指令。(在MIPS教程和P2部分，如果不对延迟槽加以说明，我们都不用考虑延迟槽)

相比于“第一个汇编程序”小节中的输出“Hello World”的程序，这里多了`syscall`输入和输出整数的用法，以及`move`, `bgt`扩展指令，在Execute页面中对照Basic列和Source

列的不同，就可以明白那条扩展指令是什么意思。利用这些扩展指令，可以简化我们编写汇编代码的工作量，并使得我们的汇编代码更加清晰。

Bkpt	Address	Code	Basic	Source
☐	0x00003000	0x24020005	addiu \$2,\$0,5	2: li \$v0,5
☐	0x00003004	0x0000000c	syscall	3: syscall # 输入一个整数，输入的数存到\$v0中
☐	0x00003008	0x00028021	addu \$16,\$0,\$2	4: move \$s0,\$v0 # 赋值，\$s0=\$v0
☐	0x0000300c	0x24110000	addiu \$17,\$0,0	5: li \$s1,0 # \$s1 用于存储累加的值，\$s1=0
☐	0x00003010	0x24080001	addiu \$8,\$0,1	6: li \$t0,1 # \$t0是循环变量
☐	0x00003014	0x0208082e	slt \$1,\$16,\$8	8: bgt \$t0,\$s0,loop_end # 这里用了一个扩展指令bgt,当\$1<\$16时，跳转到loop_end
☐	0x00003018	0x14200003	bne \$1,\$0,3	
☐	0x0000301c	0x02288820	add \$17,\$17,\$8	9: add \$s1,\$s1,\$t0 # \$s1=\$s1+\$t0
☐	0x00003020	0x21080001	addi \$8,\$8,1	10: addi \$t0,\$t0,1 # \$t0=\$t0+1
☐	0x00003024	0x08000c05	j 0x00003014	11: j loop # 无条件跳转到loop标签
☐	0x00003028	0x00112021	addu \$4,\$0,\$17	13: move \$a0,\$s1 # 赋值，\$a0=\$s1
☐	0x0000302c	0x24020001	addiu \$2,\$0,1	14: li \$v0,1 # \$v0=1，在syscall中会输出\$a0的值
☐	0x00003030	0x0000000c	syscall	15: syscall
☐	0x00003034	0x2402000a	addiu \$2,\$0,10	16: li \$v0,10 # \$v0=10
☐	0x00003038	0x0000000c	syscall	17: syscall # 结束程序

在MIPS中使用数组

下面以一个简单题目为例，说明在MIPS中如何使用数组。

题目内容：输入一个整数 $n(2 \leq n \leq 10)$ ，接下来 n 行，每行一个整数(在int范围内)，在输出的时候先输出The numbers are:，然后把这 n 个整数按照输入顺序输出，两个数字中间一个空格，行末可以有空格。

样例输入：

```
4
2
3
1
4
```

样例输出：

```
The numbers are: 2 3 1 4
```

文本代码：

```
.data
array: .space 40
# 储存这些数需要用到数组，数组需要使用10*4=40字节，
# 一个int整数需要占用4个字节，需要储存10个int整数，
# 因此array[0]的地址为0x00，array[1]的地址为0x04
# array[2]的地址为0x08，以此类推
str: .asciiz "The numbers are:\n"
space: .asciiz " "
```

```

.text
li $v0, 5
syscall    # 输入一个整数
move $s0, $v0    # $s0 is n
li $t0, 0    # $t0循环变量

loop_in:
    beq $t0, $s0, loop_in_end    # $t0 == $s0的时候跳出循环
    li $v0, 5
    syscall    # 输入一个整数
    sll $t1, $t0, 2    # $t1 = $t0 * 4
    sw $v0, array($t1)    # 把输入的数存到array+$t1的内存中
    addi $t0, $t0, 1    # $t0 = $t0 + 1
    j loop_in    # 跳转到loop_in

loop_in_end:
    la $a0, str
    li $v0, 4
    syscall    # 输出提示信息
    li $t0, 0

loop_out:
    beq $t0, $s0, loop_out_end
    sll $t1, $t0, 2    # $t1 = $t0 << 2, 即$t1 = $t0 * 4
    lw $a0, array($t1)    # 把内存中地址为array+$t1的数取出到$a0中
    li $v0, 1
    syscall    # 输出$a0
    la $a0, space
    li $v0, 4
    syscall    # 输出一个空格
    addi $t0, $t0, 1
    j loop_out

loop_out_end:
    li $v0, 10
    syscall    # 结束程序

```

这里使用了`lw`和`sw`指令，对内存进行读写操作。`lw`要求`sw`地址对齐到字(即地址必须是4的倍数),否则会产生异常。由于字符串的长度不确定，并且打印字符串(`$v0`为4)的`syscall`对`$a0`的值没有对其要求，因此在一般情况下，会把字符串的声明放在最后。同学们可以尝试把`str`的声明挪到`array`声明的前面，如果运行程序时产生了异常，则可以尝试让`str`字符串增加或减少一些字符，直到内存指令不再出现异常。

在MIPS中使用二维数组

下面再以一个简单题目为例，说明在MIPS中如何使用二维数组。

题目内容：输入两个数 m 和 n ($1 \leq m \leq 8, 1 \leq n \leq 8$)，代表有一个 m 行 n 列的矩阵，接下来 $m \times n$ 行每行一个int范围内的整数，依次输入矩阵的元素，最后输出这个矩阵

(每两个数字中间一个空格，行末可以有多个空格，最后可以有多个回车)

样例输入:

```
2
3
1
2
3
4
5
6
```

样例输出:

```
1 2 3
4 5 6
```

文本代码:

```
.data
matrix: .space 256
str_enter: .asciiz "\n"
str_space: .asciiz " "

.macro getIndex(%ans, %i, %j)
    sll %ans, %i, 3
    add %ans, %ans, %j
    sll %ans, %ans, 2
.end_macro

.text
li $v0, 5
syscall
move $s0, $v0 # 行数
li $v0, 5
syscall
move $s1, $v0 # 列数

# 这里使用了循环嵌套
li $t0, 0 # $t0是一个循环变量

in_i: # 这是外层循环
    beq $t0, $s0, in_i_end
    li $t1, 0 # $t1是另一个循环变量

in_j: # 这是内层循环
    beq $t1, $s1, in_j_end
    li $v0, 5
```

```

syscall
getindex($t2, $t0, $t1)
sw $v0, matrix($t2) # matrix[$t0][$t1] = $v0
addi $t1, $t1, 1
j in_j

in_j_end:
addi $t0, $t0, 1
j in_i

in_i_end:
# 这里使用了循环嵌套，和输入的时候同理
li $t0, 0

out_i:
beq $t0, $s0, out_i_end
li $t1, 0

out_j:
beq $t1, $s1, out_i_end
getindex($t2, $t0, $t1)
lw $a0, matrix($t2) # $a0 = matrix[$t0][$t1]
li $v0, 1
syscall
la $a0, str_space
li $v0, 4
syscall # 输出一个空格
addi $t1, $t1, 1
j out_j

out_j_end:
la $a0, str_enter
li $v0, 4
syscall # 输出一个回车
addi $t0, $t0, 1
j out_i

out_i_end:
li $v0, 10
syscall

```

注意这里使用了宏，使得程序代码更为简洁，同学们可以在Execute页面中比较Basic和Source这两列的不同。

除此以外，同学们可以在MARS编辑器中按F1快捷键，阅读MARS的帮助文档，也可以在写程序的时候阅读一下代码补全的提示信息，从中也可以找到指令集文档中没有说明的指令，阅读这些文档对同学们编写汇编程序有较大的帮助。MIPS教程中只介绍了有限的几个指令，更多教程中没有介绍的指令还等待同学们的探索。

求Fibonacci数的程序

下面给出的是求前12个Fibonacci数的汇编程序。后面会以此为例，进行程序解析。

文本代码:

```
.data
fibs: .space 48
size: .word 12
space: .asciiz " "
head: .asciiz "The Fibonacci numbers are: \n"

.text
    la $t0, fibs
    la $t5, size
    lw $t5, 0($t5)
    li $t2, 1
    sw $t2, 0($s0)
    sw $t2, 4($s0)
    addi $t1, $t5, -2

loop:
    lw $t3, 0($t0)
    lw $t4, 4($t0)
    add $t2, $t3, $t4
    sw $t2, 8($t0)
    addi $t0, $t0, 4
    addi $t1, $t1, -1
    bgtz $t1, loop
    la $a0, fibs
    add $a1, $zero, $t5
    jal print
    li $v0, 10
    syscall

print:
    add $t0, $zero, $a0
    add $t1, $zero, $a1
    la $a0, head
    li $v0, 4
    syscall

out:
    lw $a0, 0($t0)
    li $v0, 1
    syscall
    la $a0, space
    li $v0, 4
    syscall
    addi $t0, $t0, 4
    addi $t1, $t1, -1
    bgtz $t1, out
    jr $ra
```

- 1~7行: 变量声明与分配(伪指令)

- 8~14行：初始化与寄存器分配
- 16~28行：循环计算并保存Fibonacci数
- 30~47行：输出提示语句和Fibonacci数

变量声明与定义

这部分使用到的就是在MIPS架构指令集一节中初步接触到的伪指令，在该程序中这些伪指令主要用途是表示数据段和代码段的位置，并为声明的数据分配空间。

```
.data
fibs: .space 48 # "array" of 12 words to contain fib values
size: .word 12 # size of "array"
space: .asciiz " " # space to insert between numbers
head: .asciiz "The Fibonacci numbers are:\n"

.text
```

MIPS汇编程序设计

测试程序设计（本章请将开始搭建CPU时在进行阅读）